

DL Midsem Notes — Section 8

Activation Functions, Weight Initialization, Batch Normalization

1 Theory

1.1 Why Activation Functions at All

Without a non-linearity, stacking layers is pointless: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$ is just another single linear layer, no matter how many layers you stack. Non-linear activations are what let deep networks represent non-linear decision boundaries / functions at all.

Characteristics of a good activation: differentiable (a.e.), non-saturating over most of its useful range (gradient doesn't vanish), ideally zero-centered (helps optimization), computationally cheap.

1.2 Common Activations (recap + drawbacks)

Activation	Range	Main drawback
Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Saturates both ends $\Rightarrow$ vanishing gradient; not zero-centered
Tanh	$(-1, 1)$	Saturates both ends (same vanishing-gradient issue), but zero-centered
ReLU $\max(0, z)$	$[0, \infty)$	Dead neurons for $z < 0$ (zero gradient permanently)
Softmax	each in $(0, 1)$, sums to 1	Only meaningful as a multi-class output layer, not a hidden layer

Sigmoid derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, maximum value 0.25 at $z = 0$ — even at its best, gradient magnitude shrinks; stacked over many layers this compounds into vanishing gradients.

Softmax vs Sigmoid: $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ turns a vector of scores into a probability distribution over K mutually-exclusive classes (all outputs sum to 1, and increasing one score necessarily decreases the others' share). Sigmoid treats each output independently, valid range $(0, 1)$ per unit with *no* cross-normalization — use it for binary classification or multi-label problems (an example can belong to several classes at once). Use softmax when exactly one class must be chosen from many.

1.3 Weight Initialization

Why not zero (or any single constant)? Every neuron in a layer would compute the exact same output and receive the exact identical gradient — they'd stay identical forever (the "symmetry problem"), wasting the whole layer's capacity down to one effective neuron.

Xavier / Glorot init (for sigmoid/tanh): choose weight variance $\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$ (or the simpler $1/n_{in}$ variant), chosen so that the variance of activations neither shrinks nor explodes as signal passes forward (and symmetrically for gradients passing backward) through many layers.

He init (for ReLU): $\text{Var}(w) = \frac{2}{n_{in}}$. The factor of 2 compensates for the fact that ReLU zeroes out roughly half its inputs (all negative pre-activations), which would otherwise halve the forward-propagated variance at every layer if using Xavier's constant.

1.4 Batch Normalization

For each mini-batch, normalize each pre-activation: $\hat{a} = \frac{a - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, then apply a learnable scale and

shift: $z = \gamma\hat{a} + \beta$ (so the network can undo the normalization if that's actually better for a given unit).

Train vs test: during training, μ_B, σ_B^2 come from the current mini-batch. At test time there may be no batch (or batches of size 1), so a running average of μ_B, σ_B^2 collected during training is used instead — fixed statistics, not recomputed per test batch.

Benefits: reduces internal covariate shift (each layer sees a more stable input distribution as earlier layers change during training), allows higher learning rates, and has a mild regularizing effect (the per-batch noise in μ_B, σ_B^2 acts like a small perturbation).

2 Practice Questions

Q1 (PYQ-style, CS6510-5 Q4a). Why is an activation function used in a neural network? What are the characteristics of a good activation function? Explain the merits and drawbacks of the sigmoid activation function by computing its derivative.

Q2 (PYQ-style, CS6510-7 Q1b). What is the softmax activation function? How is it different from sigmoid, and when would you use softmax instead of sigmoid?

Q3 (PYQ-style, CS6510-2 Q3). A nonlinear function is defined as $f(x) = 4$ if $x \geq 0$, else -2 . A friend recommends using this as the activation in a deep network trained with the Adam optimizer. Would you follow this advice? Why or why not?

Q4 (concept check). Why does He initialization use variance $2/n_{in}$ instead of Xavier's $1/n_{in}$ (or $2/(n_{in} + n_{out})$)? What would happen if you used Xavier init throughout a deep ReLU network?

Q5 (concept check). What exactly is different about batch normalization's behaviour at train time vs. test time, and why can't test time just reuse "the same procedure" as train time?

3 Answers

A1. Activation functions introduce non-linearity — without them, any depth of stacked linear layers collapses algebraically into one linear layer (shown in Theory 1.1), so the network could only ever represent linear functions. A good activation is differentiable almost everywhere (for gradient-based learning), avoids saturating over most of its useful input range (to keep gradients informative), and ideally is zero-centered and cheap to compute.

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$. Derivative: let $s = \sigma(z)$; $\sigma'(z) = \frac{d}{dz}(1 + e^{-z})^{-1} = (1 + e^{-z})^{-2}e^{-z} = \sigma(z) \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z))$. **Merit:** smooth, bounded output interpretable as a probability.

Drawbacks: saturates for large $|z|$ where $\sigma'(z) \rightarrow 0$ (vanishing gradient, compounds badly across many layers); output is not zero-centered (all-positive outputs bias the sign of gradients in the next layer); involves an expensive exponential.

A2. Softmax converts a vector of raw scores into a probability distribution over K classes: $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$, with all outputs summing to 1. Sigmoid produces an independent $(0, 1)$ value per unit with no cross-normalization. Use **softmax** for single-label multi-class classification (exactly one correct class, competing probabilities); use **sigmoid** for binary classification or multi-label classification (an input may belong to several classes simultaneously and each unit's probability is judged independently).

A3. No, don't use it. This f is a step function: its derivative is 0 everywhere except an undefined jump at $x = 0$. Every weight upstream of a neuron using this activation receives a gradient of exactly 0 almost everywhere (via the chain-rule factor $\varphi'(a)$), so no gradient-based optimizer — however sophisticated its adaptive scaling, Adam included — can update those weights; Adam's momentum and per-parameter scaling only rescale an existing gradient signal, they can't manufacture one that isn't there. The network simply won't learn through this activation.

A4. ReLU zeroes out roughly half its inputs (every negative pre-activation), so on average only about half the units in a layer pass a non-zero signal forward. If you kept Xavier's variance (calibrated assuming the activation preserves signal symmetrically, as sigmoid/tanh roughly do near 0), the effective forward-propagated variance would be cut roughly in half at every layer, compounding into vanishing activations (and correspondingly vanishing gradients) in deep ReLU networks. He initialization doubles the variance ($2/n_{in}$) to exactly compensate for that expected halving, keeping the signal's variance stable layer to layer.

A5. At train time, μ_B, σ_B^2 are computed fresh from the current mini-batch, so the normalization is a live, mini-batch-dependent operation. At test time you often see one example at a time (or arbitrary batch sizes), so there is no meaningful "batch statistic" to compute on the fly, and using single-example statistics would make normalization at test time noisy or degenerate (e.g. dividing by a variance of 0 for a batch of size 1). Instead, a running average of μ_B, σ_B^2 accumulated across many mini-batches during training is stored and reused, unchanged, at test time — a fixed, deterministic transform rather than one recomputed per test batch.